

Relatório — Painel de Controle Universal Multi-Harness

Status: MVP implementado e validado na branch `feature/painel-controle-multi-harness`

Data: 2026-08-13 **Ponto de retorno:** branch dedicada, nenhum arquivo do pipeline existente (skills, scripts, SPECs) foi alterado. Não fazer merge da branch = main permanece intocado. **Plano**

original: `melhorias/plano-painel-controle-multi-harness.md`

1. O que foi pedido

Dar “cara” (frontend) à Fábrica de Materiais de Comunicação, permitindo que o usuário escolha o provedor/LLM e as credenciais, com todos os artefatos gerados dentro de uma pasta que o próprio usuário define — sem banco de dados de conteúdo — e sem reescrever nenhuma skill/SPEC existente.

2. O que foi implementado

Novo pacote `painel/` (Python, FastAPI), 26 arquivos, ~2.830 linhas, em 5 commits sequenciais, cada um validado por testes reais antes do próximo passo:

#	Componente	Commit
1	Plano salvo em <code>melhorias/</code> (.md + .pdf)	7bac977
2	Scaffold + índice SQLite + workspace + cofre de credenciais + adaptadores de harness + job runner	c37a5e6
3	API FastAPI (<code>workspace/credenciais/projetos/jobs</code>)	09058ae
4	Frontend mínimo (wizard + dashboard)	b12e3b9
5	README com instruções e limitações	7d7a9bf

Arquitetura entregue

- **Workspace** (`painel/workspace.py`): usuário registra a pasta onde quer que todos os projetos/artefatos sejam salvos. Validada (não pode ser vazia, não pode ser a pasta interna do painel, não pode ser um arquivo).
- **Cofre de credenciais** (`painel/vault.py`): API keys/tokens de cada harness, criptografados (Fernet) em `~/.fabrica-painel/vault.enc` — nunca dentro do workspace do usuário. Provado por teste que o segredo não aparece em claro no arquivo em disco.
- **Registry de adaptadores de harness** (`painel/harness_adapters/`): o mecanismo que resolve “universal”. Três adaptadores hoje — `echo` (dry-run, seguro para smoke test real), `claude-code`

(claude -p), opencode (opencode run) — cada um só monta comando/env/cwd; nenhuma skill ou SPEC da Fábrica foi tocada ou reescrita.

- **Job runner** (painel/jobs.py): spawna o adaptador escolhido em subprocess real, roda em thread de segundo plano, atualiza status (pending → running → done/error) e log no índice SQLite (~/.fabrica-painel/painel.db) — **nunca o conteúdo gerado**, que fica só no workspace.
- **API FastAPI** (painel/main.py): endpoints REST ligando tudo — /api/workspaces, /api/credentials, /api/harnesses, /api/projects (lê/escreve config_projeto.json direto na pasta do usuário, sem duplicar em banco), /api/jobs.
- **Frontend mínimo** (painel/static/): página única (HTML/JS vanilla, sem framework/build step) servida pelo próprio backend — wizard de workspace/credencial/harness/modelo, criação de projeto e disparo/ acompanhamento de job.

3. Como foi validado (evidência, não opinião)

- **37 testes automatizados** (pytest), todos verdes a cada passo do plano antes de avançar para o próximo — nenhum passo seguiu com teste vermelho.
- Nada foi validado só por mock no caminho principal: o adaptador echo roda **subprocess real** em todo teste que o exercita (inclusive via HTTP real com TestClient), criando arquivo de verdade em disco.
- **Smoke test manual end-to-end** com servidor uvicorn real e requisições curl reais (não TestClient): registrar workspace → criar projeto → disparar job → job concluído (status: done, exit_code: 0) → confirmado que config_projeto.json e o artefato do job ficaram dentro da pasta do usuário, e que painel.db/logs ficaram isolados em ~/.fabrica-painel, fora do workspace.

```
GET /api/projects?workspace_path=<workspace>
[{"slug":"smoke-e2e","config":{"..."},"manifesto":null}]
```

```
GET /api/jobs/1
{"status":"done","exit_code":0,...}
```

```
ls <workspace>/smoke-e2e/
config_projeto.json  smoke_marker.txt
```

4. O que NÃO foi feito (limitações documentadas em painel/README.md)

1. **Descoberta de skills para workspace externo ao repo não está resolvida.** Os adaptadores claude-code/opencode rodam com cwd = pasta do projeto no workspace escolhido; se esse workspace estiver fora deste repositório, o harness não vê .claude/skills/, SPEC_COMANDOS.md, AGENTS.md. É a decisão de arquitetura mais importante pendente antes de uso real — três caminhos possíveis estão descritos no README (workspace como subpasta do repo, skills instaladas em nível global do harness, ou flag de project-root explícito).
2. **claude-code e opencode não foram exercitados com subprocess real** — só construção de comando testada (mockada), de propósito: evita custo de LLM e risco de recursão de agente dentro desta própria sessão. Precisa do binário instalado + credencial real do usuário para validar de verdade.

3. **Fluxo one-shot substitui a entrevista conversacional do /esbocar** — o formulário entrega tudo de uma vez; perde a adaptação dinâmica de perguntas de acompanhamento que a REGRA 3 do AGENTS.md prevê para a interação humana.
4. Cofre de credenciais é single-user (sem rotação de chave/HSM); API sem autenticação (assume uso só em localhost); job runner é thread simples, sem fila com prioridade/limite de concorrência; frontend sem polimento visual — é wizard funcional, não produto finalizado.

5. Próximos passos recomendados (se decidirem seguir)

1. Decidir a estratégia de descoberta de skills para workspace externo (item 1 da seção 4) — é o bloqueador real antes de rodar claude-code/ opencode de verdade a partir do painel.
2. Validar manualmente o adaptador claude-code (ou opencode) com credencial real do usuário, fora desta sessão, para fechar a lacuna do item 2.
3. Decidir se o formulário one-shot de criação de projeto precisa replicar mais fielmente a lógica de analista-insumos/diretor-de-arte antes de disparar produção real.

6. Reversibilidade

Toda a implementação está isolada em feature/painel-controle-multi-harness, em uma pasta nova (painel/) que não modifica nenhum arquivo do pipeline existente. Para descartar: não fazer merge (e opcionalmente apagar a branch). Para retomar: `git checkout feature/painel-controle-multi-harness`.